DSA-301 HackerRank Challenge Editor

Two things live here:

1. `editor.py` - pushes challenge content to HackerRank from a markdown file. It edits existing challenges by matching on their title, so it never creates duplicates.
2. `make_md.py` - generates those markdown files. It holds a statement and a reference solution for each problem, and **computes** the test data by running the solution.
3. `problem_links.py` - the URL of the original problem for each challenge. 214 of the 230 are LeetCode's or GeeksforGeeks'; `make_md.py` appends a link to each statement. See section 4.

Together they replace 230 challenges' worth of placeholder content with real problems and correct test data.

0. Run it: the whole sequence

Every command runs from this folder. If you want only the commands and the order they go in, read `RUNBOOK.md` / `DSA301-Runbook.pdf` instead - it is five pages and covers the same sequence, plus a troubleshooting table.

Step 1 -- install, once.

```
python -m venv venv
venv\Scripts\python.exe -m pip install -r requirements.txt
```

Step 2 -- start Chrome with debugging. Every session, and leave it open.

```
"%LOCALAPPDATA%\Google\Chrome\Application\chrome.exe" --remote-debugging-port=9222 --user-data-dir="%USE
    RPROFILE%\chrome-debug-profile"
```

Log in to HackerRank in that window the first time. It must be this separate profile -- see section 2.

Step 3 -- check what would change, writing nothing.

```
venv\Scripts\python.exe editor.py DSA301_LEC01-10.md
```

Read the three counts it prints. `not matched` entries are skipped and never written.

Step 4 -- push, one file at a time.

```
venv\Scripts\python.exe editor.py DSA301_LEC01-10.md --apply --tests --samples 3
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --apply --tests --samples 3
venv\Scripts\python.exe editor.py DSA301_LEC21-30.md --apply --tests --samples 3
venv\Scripts\python.exe editor.py DSA301_LEC31-40.md --apply --tests --samples 3
venv\Scripts\python.exe editor.py DSA301_LEC41-50.md --apply --tests --samples 3
```

`--samples 3` because these files are generated with `--link-only`, where the visible samples are the only statement of the input format. On files generated without it, the default of 2 is fine - see section 4.

Order does not matter, and re-running is safe: matching is by title and the edit happens in place, so a second run reports `already correct` and does nothing.

Expect roughly **11 seconds per challenge** -- a page load, the field writes, a save, then a read-back comparison. That is about 8 minutes for the 42-problem file and around 45 minutes for all five. Chrome must stay open throughout.

Step 5 -- after changing any problem, regenerate the markdown.

```
venv\Scripts\python.exe make_md.py
```

Then push again. Edit the Python, never the markdown -- see section 7.

Doing less than everything

```
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --apply
```

Statements and names only, no test cases.

```
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --only "LC94" --apply --tests
```

A single challenge. Use this for the first real write, and to resolve a duplicate pair once you have chosen which copy is canonical.

```
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --apply --tests --samples 3
```

Show students three sample cases instead of the default two.

Rebuilding the guides as PDFs

```
venv\Scripts\python.exe -m pip install reportlab
venv\Scripts\python.exe md2pdf.py README.md DSA301-Editor-Guide.pdf
venv\Scripts\python.exe md2pdf.py RUNBOOK.md DSA301-Runbook.pdf
```

1. Why this exists

The DSA-301 challenges were originally uploaded from generated markdown. The result, measured across all 302 live challenges:

symptom	measurement
Statements were templates	27 distinct skeletons; the top 9 covered 212 challenges
Constraints were near-identical	227 of 238 shared one block
Test data was fabricated	**215 challenges shared only 4 distinct datasets**
No visible samples	every case was flagged `sample: false`
Re-runs created duplicates	12 redundant copies, 20 slugs carrying a collision counter

The third row is the fatal one. `Minimum cost to connect ropes`, `LC1353 Maximum Number of Events` and `Rearrange characters` all carried the same five test cases of 4, 6, 8 and 14 bytes. One input cannot be correct for three unrelated problems, so the expected outputs were never computed from a working solution.

Two root causes, both fixed here:

- **Nothing recorded what was created.** The old script hit `/challenges/create` every iteration and never captured the resulting ID, so re-running produced duplicates instead of edits.
- **Verification was fake.** It accepted a field if the text was non-empty. On an *edit* page every field is already populated, so that check passes even when the write silently failed.

2. Setup

Python packages (once):

```
python -m venv venv
venv\Scripts\python.exe -m pip install -r requirements.txt
```

Chrome with remote debugging (each session):

```
"%LOCALAPPDATA%\Google\Chrome\Application\chrome.exe" --remote-debugging-port=9222 --user-data-dir="%USE
    RPROFILE%\chrome-debug-profile"
```

Log in to HackerRank once in that window; the profile remembers it.

> **The separate profile is not optional.** Chrome 136 and later refuse to open the debugging port when `--user-data-dir` points at your normal profile. Your everyday Chrome shortcut will not work for this.

Authentication is that Chrome session. Every request runs inside the page with `credentials: 'same-origin'`, so **the tool never handles your password.**

3. Using `editor.py`

```
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md                dry run
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --apply         write statements
venv\Scripts\python.exe editor.py DSA301_LEC11-20.md --apply --tests  also replace test cases
```

flag	effect
(none)	Dry run. Shows what matched and what would change. Writes nothing
`--apply`	Writes the name and the four statement fields
`--tests`	Also replaces test cases from the `Sample Input/Output` blocks
`--samples N`	How many cases students can see. Default 2
`--only "LC94"`	Just the entries whose title contains this

A dry run reports three groups:

```
matched and already correct : 0
matched and needs work      : 42
not matched                 : 0
```

`not matched` is the one to read. It means either no live challenge has that title, or **two** do - and in the second case `editor.py` refuses to guess and skips it, naming both IDs so you can pick one with `--only`.

Run with no arguments to print the usage and the expected markdown format.

4. Using `make_md.py`

```
venv\Scripts\python.exe make_md.py
```

Regenerates all five markdown files. It first checks every reference solution against answers worked out by hand:

```
checking reference solutions against hand-worked answers...
  DSA301_LEC11-20.md     60 checks, 0 failed
  ...
```

If any check fails, nothing is written. That gate has already caught real mistakes - a mis-transcribed Group Anagrams answer, and a Task Scheduler value I had computed wrongly by hand. In both cases the code was right and the expectation was wrong, which is exactly what the gate is for.

Then it resolves the source link for every problem:

```
resolving source links...
  DSA301_LEC11-20.md      30 LeetCode,  8 GeeksforGeeks,  4 own
  ...
  total                  163 LeetCode, 51 GeeksforGeeks, 16 own
```

That is a second gate, and it also writes nothing if it fails - see below.

Source links

Most of these problems are not ours. An `LC<n>` in the title means LeetCode problem *n*; a `(GfG)` suffix means a GeeksforGeeks problem. `make_md.py` appends the original's URL to the end of each statement:

```
Original problem: [LeetCode 94 - Binary Tree Inorder Traversal](https://leetcode.com/problems/binary-tre
    e-inorder-traversal/).
Same problem, but solve it to the input and output format below -- this
challenge reads standard input and prints to standard output rather than
filling in a function.
```

The second sentence matters. LeetCode and GeeksforGeeks hand you a function signature; this grader reads stdin. Without that line a student who follows the link writes a `Solution` class and scores zero on every case.

Suppress the links entirely with:

```
venv\Scripts\python.exe make_md.py --no-links
```

which reproduces the markdown byte-for-byte as it was before links existed.

The three modes

mode	statement	input / output format, constraints	problems affected
(default)	full write-up **+** link	full	all 230
`--no-links`	full write-up	full	all 230
`--link-only`	**just the link**	one-line placeholder	the 214 with a source

`--link-only` turns a challenge into a pointer. The statement becomes one line:

```
Solve this problem here: [LeetCode 200 - Number of Islands](https://leetcode.com/problems/number-of-isla
    nds/)
```

The student reads the problem at the source. It drops about 2,300 lines of prose across the five files.

The other three boxes cannot be left empty. That was the first design, and HackerRank rejected it. The edit form validates and saves the four statement fields **as a unit**, so three blank boxes refuse the entire submission - the read-back showed even `problem_statement` unchanged, though it had text:

```
id=592083   Classwork: LC200 Number of Islands
   differs: problem_statement, input_format, constraints, output_format
   FAILED verification: problem_statement, input_format, constraints, output_format
```

Measured, not assumed, and the reason `editor.py` verifies by read-back rather than trusting the save: "is the box non-empty" would have *passed* here, because the old text was still sitting in the field.

So the three boxes get the shortest honest line instead, from `LINK_ONLY_INP` and friends in `problem_links.py`. The same challenge, same command, after that change:

```
id=592083   Classwork: LC200 Number of Islands
   differs: problem_statement, input_format, constraints, output_format
   saved and verified
   test cases replaced: 8
   samples: 2 marked as sample
```

Both halves of that experiment are on the record because together they pin the cause to the blank fields and nothing else - same challenge, same flags, one variable changed.

```
Input Format    Standard input, in the form shown by the sample cases below.
Output Format   Standard output, in the form shown by the sample cases below.
Constraints     As stated in the linked problem.
```

Set any of those to `""` if you want to retry the empty-field experiment.

`LC200` (challenge 592083) is the one challenge live in link-only form; it is the worked example if you want to see what a student sees before pushing the rest.

The 16 problems written for this curriculum are untouched by the flag - there is nowhere to send a student for *Reverse a singly linked list iteratively*, so they keep their full write-up in every mode. `link_only_for()` returns `""` for them and `emit()` falls through to the normal path.

What `--link-only` does not remove: the sample cases. They are still emitted and still uploaded by `--tests`, and that is what keeps these challenges solvable. LeetCode hands out a function signature; this grader reads stdin. With the input format field empty, the two visible samples become the *only* statement of the I/O contract - a student infers "first line `4 5`, then the grid" from the example rather than from prose. So in link-only mode `--samples` is no longer cosmetic. Consider raising it:

```
venv\Scripts\python.exe editor.py DSA301_LEC41-50.md --apply --tests --samples 3
```

If the intent is instead that students solve on LeetCode and never submit here, the test data does not matter and you can push with `--apply` alone.

The links are a checked-in table, not a guess. `problem_links.py` holds two:

table	keyed by	source
`LEETCODE`	problem number	LeetCode's own list, `leetcode.com/api/problems/all/`
`GFG`	our full title	GeeksforGeeks' practice search API

Neither is fetched at generation time, so `make_md.py` still runs offline and a link cannot silently change under you. To re-check them against the live sites:

```
venv\Scripts\python.exe problem_links.py --verify
```

That is the only thing in this repo that touches the network, and it is a mode you run on purpose. It confirmed all 214 links on 2026-08-20. The two sites need different checks: LeetCode publishes its whole problem list as JSON, so one request confirms every number, slug, title and premium flag at once. GeeksforGeeks practice pages are a single-page app that answers **HTTP 200 for any slug, real or not** - a status code proves nothing there, so the check reads the server-rendered `<title>` instead and compares it to the recorded name. Their *articles* do return a real 404, so for the three article links the status code is enough.

Both had to be tables because **neither slug can be derived from our titles**:

- Our LeetCode titles are sometimes shortened. `LC105 Construct Binary Tree from Preorder and Inorder` is really *... and Inorder **Traversal***, and `LC1584 Min Cost to Connect All Points (MST)` carries a parenthetical that is not part of the name. Slugifying our titles gives a 404 for **7 of the 163**: `LC105`, `LC106`, `LC235`, `LC255`, `LC889`, `LC1008`, `LC1584`.
- GeeksforGeeks practice slugs carry arbitrary numeric suffixes - `minimum-platforms-1587115620`, `rod-cutting0840`, `cutted-segments1642`. There is no rule to generate those.

Three GeeksforGeeks entries point at an article rather than a practice problem, because no practice equivalent exists: *0/1 BFS*, *Minimum subset-sum difference*, *Rearrange characters*. Five LeetCode problems are premium-only (`LC161`, `LC255`, `LC269`, `LC285`, `LC333`); their labels say so, so a student who hits the paywall knows why.

`link_for()` **raises** on a title that names a source with no entry. A new `LC999` problem is a missing mapping, not a problem without a source, so generation aborts rather than quietly emitting a statement with no link - the same policy as the sanity gate, and it is what catches a mistyped problem number.

The 16 problems with no link are the ones actually written for this curriculum: *Reverse a singly linked list iteratively*, *Compute factorial and nCr recursively*, and so on.

Adding or changing a problem

Each problem is a dict in `make_md.py`, `dp_problems.py` (LEC 31-40) or `graph_problems.py` (LEC 41-50):

```
dict(title="Classwork: LC94 Binary Tree Inorder Traversal",
     stmt="Given the root of a binary tree, return the inorder traversal ...",
     inp=TREE_IN,
     out=SEQ_OUT,
     cons=CONS,
     solve=lambda s: inorder(s),
     cases=TREES),
```

- `title` must match the live challenge name. It is the join key.
- `solve` takes the input as a string and returns the expected output as a string. **The sample outputs are computed by running this** - they are never typed by hand. That single rule is what prevents a repeat of the original failure.
- `cases` are input strings. **Put your clearest cases first**: the leading ones become the samples students see. Edge cases belong further down - an empty input printing an empty line makes a useless first example.
- Add at least one entry to the matching `SANITY` list, with an answer you worked out yourself rather than by running the code.

Two rules the statements follow

State the problem, not the method. The statement says what is given, what to print, and what the conventions are. It does not say how to solve it. "Sort by finish time and take greedily" is the answer, not a specification.

Pin down anything ambiguous. Where a problem admits several valid answers the statement demands one: the lexicographically smallest topological order, groups sorted within and between, ties broken by the smaller value. Otherwise correct solutions fail on ordering.

Some titles name a required technique - *Iterative BST search*, *Morris inorder traversal (O(1) space)*, *Longest Increasing Subsequence in O(n log n)*. That is deliberate: it is a **constraint**, not a hint, and for eight problems it is the only thing distinguishing them from another challenge in the same curriculum. Removing it would collapse them into duplicates.

5. What the files contain

file	problems	test cases	hand-worked checks	LC	GfG	ours
DSA301_LEC01-10.md	35	280	54	19	4	12
DSA301_LEC11-20.md	42	387	60	30	8	4
DSA301_LEC21-30.md	48	406	64	32	16	0
DSA301_LEC31-40.md	54	432	98	40	14	0
DSA301_LEC41-50.md	51	402	85	42	9	0
total	**230**	**1,907**	**361**	**163**	**51**	**16**

Source of truth is `DSA_101_Curriculum.xlsx`, sheet `DSA_301`, columns **Classwork** and **Homework** only. That sheet defines 300 assignment slots which reduce to 276 distinct challenges, because 18 problems are deliberately

reused across lectures.

LEC-1 and LEC-2 are deliberately absent. Those 10 lambda challenges already had real statements and unique test data - they were the only part of the original set made properly. Replacing them would have been a downgrade.

6. How it works

Writing the statement

The edit page is `/administration/challenges/edit/<id>/details`. That route was found by observing an actual row click, **not** by probing URLs: the admin panel is a single-page app that returns HTTP 200 for every `/administration/*` path, so a 200 there proves nothing.

The four statement boxes are **CodeMirror** editors, in this order:

index	field
0	problem statement
1	input format
2	constraints
3	output format

They are set with `cms[i].CodeMirror.setValue(text)`. That ordering was confirmed by reading each editor's existing contents and matching them against the REST API, not by guessing. If the page does not present exactly four editors, `editor.py` **skips that challenge** rather than writing fields into the wrong boxes.

Then it clicks `Save Changes`, re-fetches `/rest/administration/challenges/<id>`, and compares every field against what it sent. A mismatch is reported per field and counted as a failure.

Writing the test cases

1. Build a zip containing `input/inputNN.txt` and `output/outputNN.txt`.
2. `POST` it to `/rest/administration/challenges/<id>/test_cases/upload_zip` as multipart, form field `zip_file`.
3. Count the cases before and after, and report which happened. Upload **replaces** rather than appends - measured, not assumed.
4. Flag the first `--samples N` cases as visible samples with a `PUT` to `/rest/administration/challenges/<id>/test_cases/<ids>`.

Step 4 is easy to miss: a zip upload always lands with every case marked `sample: false`, so without it students see no examples at all.

The CSRF requirement

HackerRank's admin is a Rails app. Every non-`GET` request needs the CSRF token from `<meta name="csrf-token">`, sent as the `X-CSRF-Token` header and as the form field named by `<meta name="csrf-param">`. Reads work without it, which is why a missing token shows up as a bare `HTTP 403` on upload only.

7. Things that will bite you

IDs are the only stable handle. Names change - HackerRank appends `1`, `2`, `3` on collision - and slugs get `-1` suffixes. `editor.py` matches on the title because your markdown has no IDs, which is why ambiguous titles are

skipped rather than guessed.

Slugs are immutable. A full field rewrite left `classwork-lc94-binary-tree-inorder-traversal` untouched. The 20 slugs carrying collision counters are permanent. They are cosmetic - students are not graded on the URL.

There is no way to delete a challenge. The Details tab has no destructive control; Settings offers only `Disable Compile & Test`, `Disable Custom Testcase` and `Disable Submissions`. The single `DELETE` in the admin JavaScript removes a *contest association*, not a challenge. Combined with immutable slugs, this means editing in place is the only option - recreating is not available.

Duplicate pairs need a human decision. About 18 problems match two live challenges each. `editor.py` skips them and prints both IDs; pick the canonical one and target it with `--only`.

`make_md.py` **overwrites the markdown files.** Edit the Python, not the markdown. If you hand-edit a `.md` file, the next `make_md.py` run discards it.

8. What is not done

- **9 stateful problems** - `LC146 LRU Cache`, `LC295 Median from Data Stream`, `LC173 BST Iterator`, `LC355 Design Twitter`, `LC895 Max Frequency Stack`, `LC432 All O'one`, `LC703 Kth Largest in a Stream`, browser history, sorted stream. These do not fit an input-to-output contract and need an operation-sequence format designed once and reused.
- **~17 problems are in the sheet but not live** - `LC509`, `LC377`, `LC279`, `LC120` and others. The write-ups exist; they need **creating**, which `editor.py` deliberately does not do.
- **~18 duplicate pairs** await the decision above.
- **6 sheet entries are not coding problems** (`Write-up: a coin system where greedy fails`, `Classify five mixed problems by technique`).
- **Editorials are empty.** HackerRank has a per-challenge Editorial field, which is where approach explanations belong - visible after an attempt, not before. `editor.py` does not write it yet.
- **Only `LC94` has been submission-tested** end to end, with a real C++ solution against the live grader (`solutions/LC94_inorder.cpp`). It passed. The other 229 are verified against hand-worked answers, not against the grader.

9. Files

```
editor.py                   pushes markdown to HackerRank
make_md.py                  generates the markdown; LEC 01-10 and 11-30 problems
dp_problems.py              LEC 31-40 problems (dynamic programming)
graph_problems.py           LEC 41-50 problems (graphs, backtracking, two pointers)
problem_links.py            LeetCode and GeeksforGeeks URL per challenge
DSA301_LEC01-10.md          generated -- 35 problems
DSA301_LEC11-20.md          generated -- 42 problems
DSA301_LEC21-30.md          generated -- 48 problems
DSA301_LEC31-40.md          generated -- 54 problems
DSA301_LEC41-50.md          generated -- 51 problems
requirements.txt            three packages
solutions/LC94_inorder.cpp  worked example of a student solution
backup-all-challenges.json  snapshot of all 302 challenges before any edits
README.md                   this guide
RUNBOOK.md                  the commands only, in order
DSA301-Runbook.pdf          generated from RUNBOOK.md
DSA301-Editor-Guide.pdf     generated from README.md
```

If you share this

Include everything above **except**:

- `venv\` - 91 MB, machine-specific. The recipient runs the two setup commands.
- `backup-all-challenges.json` - a snapshot of your own live account. Not needed to use the tool, and worth keeping private.
- `__pycache__\` - regenerated automatically.

`make_md.py` reads the curriculum workbook only if you ask it to reconcile against the sheet; generating the markdown does not need it.